SFA – Architecture and Data Pipeline

Technology Stack

Layer	Technology	Details
Language	Python 3.11+	Core package: `organic_market_agent` (~13,900 lines)
Database	PostgreSQL 15	Docker (dev port 5433), SQLAlchemy 2.x ORM, Alembic migrations
Admin UI	Flask + Jinja2	Hebrew RTL, 25+ templates, port 5001
HTTP collection	httpx + BeautifulSoup4	Async-capable, retry logic, TLS handling
Scheduler	System cron → Python runner	Self-gating pipeline, daily runs
Publishing	FTPS → uPress WordPress	Custom TLS session reuse (ftputil, paramiko)
Charts	Chart.js (CDN)	Admin dashboard visualizations
Testing	pytest (127+ tests)	15 test files, `@pytest.mark.upress` for live server tests
Code quality	black, ruff	Linting and formatting

The Core Pipeline: Collect → Parse → Normalize → Aggregate → Publish

The entire SFA data flow is a sequential five-stage pipeline. Each stage produces structured output that feeds the next stage. All stages operate on PostgreSQL as the single source of truth – no intermediate file formats.

```
Raw web pages / APIs
      ↓
  [1] COLLECTORS   — HTTP fetch, retry, dedup
      ↓ raw_assets
  [2] PARSERS      — HTML/JSON extraction
      ↓ raw_extracted_items
  [3] NORMALIZER   — 8-stage data-driven pipeline
      ↓ normalized_observations
  [4] AGGREGATOR   — Statistics + QA quality gate
      ↓ daily_aggregates
  [5] PUBLISHER    — JSON/HTML artifacts + FTPS upload
      ↓
  WordPress public price index
```

Stage 1: Collectors

Module: `organic_market_agent/collectors/engine.py`

Class: `CollectorEngine`

The collector fetches raw HTTP data from each registered source. Key behaviors:

- **Per-source fetch profiles** define: collector type, parser type, HTTP headers, schedule cron, payload
- **Checksum deduplication:** Raw assets with identical content checksums are skipped (no redundant processing)
- **Retry logic:** Failed fetches retry with exponential backoff
- **Source types:** EasyFarm API (JSON), StandaloneHTML (scraped HTML), GovtBenchmark (reference pricing APIs)

Output: `raw_assets` table rows – saved raw HTTP responses with fetch metadata

---

Stage 2: Parsers

Module: `organic_market_agent/parsers/engine.py`

Class: `ParserEngine`

Parsers extract structured item data from the raw HTML/JSON content saved by collectors. There are 3 parser implementations, each targeting a specific source type:

- **EasyFarm parser:** Processes the EasyFarm platform API response (JSON) – used by several farm shop sources
- **StandaloneHTML parser:** General-purpose HTML extraction for farm websites with custom layouts
- **GovtBenchmark parser:** Processes Israeli government reference pricing data (calibration reference only)

Output: `raw_extracted_items` rows – each with: `raw_name` (Hebrew product name as found on the site), `raw_price_str`, `raw_unit`, `raw_qty_str`, `source_id`

Status after parsing: `extracted` → transitions through normalizer

Stage 3: Normalizer – The Core Innovation

Module: `organic_market_agent/normalizer/engine.py`

The normalizer is SFA's most complex and valuable component. It processes each `raw_extracted_item` through an **8-stage pipeline**, all driven by data stored in PostgreSQL – meaning the normalization logic can be updated without code changes.

The 8 Stages

Stage 1 – Scope Skip

- Filters out non-food and out-of-scope items using 301 pattern rules
- Rule types: `exact` (exact match), `prefix`, `contains`, `regex`
- Categories: `grocery` (packaged foods), `dry_grocery`, `donation`, `cleaning`
- Items matching a scope-skip rule: status → `ignored`
- Items not matching scope-skip: proceed to Stage 2

Stage 2 – Alias Resolution

- Maps raw Hebrew product names → canonical product catalog using 232 alias mappings
- Three alias types in priority order:

1. `exact` – exact string match (highest priority)
2. `global` – normalized/cleaned string match
3. `substring` – partial match (lowest priority, used when no exact match exists)

- Items successfully mapped: status → proceed to Stage 3
- Items not mappable: status → `unresolvable` (admin review queue)

Stage 3 – Organic Flag

- Detects organic certification marks in the raw name
- Sets `is_organic` flag on the observation
- Does not filter – informational only

Stage 4 – Price Parse

- Extracts a numeric price value from `raw_price_str`
- Handles: currency symbols, comma/period decimals, range notation (″₪15-20″ → midpoint), text suffixes
- Items with unparseable prices: status → `unresolvable`

Stage 5 – Unit Resolution

- Identifies the measurement unit from the raw unit string
- Maps to canonical units: `kg`, `unit`, `bunch`, `basket_small`, `pack`
- Uses `measurement_units` table for normalization

Stage 6 – Quantity Extraction

- Extracts the purchase quantity from `raw_qty_str`
- Handles: ″500g″ → 0.5kg, ″5 units″ → 5, implicit quantities

Stage 7 – Price Normalization to ₪/unit

- Converts all prices to a common unit: ₪/kg equivalent
- Uses `unit_conversions` table (e.g., 1 bunch = 0.5 kg, 1 unit zucchini = 0.35 kg)
- Products with `unit_type = kg`: no conversion needed
- Products with weight-based units: apply conversion factor
- Baskets and CSA items: treated independently (see Stage 8)

Stage 8 – Basket Check

- CSA baskets are treated as independent products, not decomposed
- Basket products (`is_basket_product = true` in catalog) are normalized as a single item
- Price per basket is the normalized value – no per-kg decomposition in V1

Output: `normalized_observations` rows with:

- `product_id` → canonical catalog entry
- `price_shekel_per_unit` → normalized ₪/kg equivalent
- `measurement_unit_id` → resolved unit

- `quantity` → resolved quantity
- `confidence_score` → 0.0–1.0 (based on alias type, unit resolution quality)
- Status → `normalized`

Stage 4: Aggregator and QA Engine

Module: `organic_market_agent/aggregator/engine.py`

QA Module: `organic_market_agent/aggregator/qa_engine.py`

The aggregator computes daily statistics from normalized observations and applies a quality gate before marking data as publishable.

Daily Aggregation

For each product with observations on a given date:
- `avg_price` – mean of all ₪/kg values
- `std_dev` – standard deviation
- `count` – number of observations
- `count_by_source` – JSON breakdown of count per source (for transparency)
- `staleness_level` – computed from observation age: `ok` (current) / `warning` (3+ days) / `stale` (8+ days)

Weekly snapshots are also generated for trend tracking.

QA Engine Quality Gates

The QA engine blocks publication when data quality is insufficient:

Minimum source requirement: Aggregation only runs when ≥2 community sources have contributed observations for the day. A price from a single farm is not published as a "market price."

Outlier detection:
- **2-source spread rule:** If only 2 sources have data and `|price1 - price2| > 50%` of the lower price → flagged, publish blocked
- **Multi-source σ rule:** If σ > ₪2 per kg across all observations for a product → flagged, admin review

Observation flags: Individual observations can be flagged by the admin (`hide`, `review`) without removing them from the database. Hidden observations are excluded from aggregation.

Stage 5: Publisher

Module: `organic_market_agent/publisher/engine.py`

FTPS Module: `organic_market_agent/publisher/ftps_upload.py`

The publisher builds versioned public artifacts and uploads them to the production WordPress server.

Artifacts Generated

Each publish run creates timestamp-versioned files plus fixed-name copies:

Artifact	Purpose
`public_report-{ts}.json`	Machine-readable price index with full metadata
`public_report-{ts}.html`	Standalone HTML viewer (not used in WordPress embed)
`public_report_body-{ts}.html`	**WordPress embed fragment** – scoped `.sfagent-*` CSS + price table
`manifest.json`	Pointer to current artifacts + versioning + staleness
`manifest_last_good.json`	Fallback copy of last successful manifest

Manifest v2 Schema

```
{
  "schema_version": "2",
  "artifact_version": "20260423-060000",
  "staleness_days": 0,
  "staleness_level": "ok",
  "artifacts": {
    "public_report": "public_report-{ts}.json",
    "public_report_body": "public_report_body-{ts}.html"
  },
  "fixed_names": {
    "public_report.json": "public_report-{ts}.json",
    "public_report_body.html": "public_report_body-{ts}.html"
  },
  "upload_base": "https://www.nimrod.bio/wp-content/uploads/market"
}
```

FTPS Upload Architecture

Upload to uPress uses a custom `ReusedSessionFTP_TLS` subclass. This is not a trivial implementation choice – standard FTPS clients fail on uPress's server because the TLS session is expected to be reused between control and data connections. The custom implementation handles this correctly, preventing `425 Can't open data connection` errors.

Upload path: `wp-content/uploads/market/` on uPress server s887.

After successful upload: optional ezCache purge (WordPress cache invalidation) and public manifest verification (HEAD request to confirm artifact is accessible).

CLI Entry Points

The entire pipeline is accessible via `python3 -m organic_market_agent`:

Command	Action
`run_ingestion`	Collectors + parsers
`run_normalize`	Normalizer stage
`run_aggregate`	Aggregation + QA
`run_publisher`	Build artifacts + optional FTPS upload
`run_publisher --upload`	Build + upload
`run_upload`	FTPS upload only (reads manifest for file list)
`catalog_renormalize`	Re-queue unresolvable items for re-normalization
`full_data_refresh`	Reset all items to `extracted`, re-run pipeline
`run_admin`	Start Flask admin UI (127.0.0.1:5001)

Scheduling Architecture

Production (waldhomeserver):

- systemd unit `sfa-admin` runs the admin UI

- Cron scheduler triggers daily pipeline: ingestion → normalize → aggregate → publish → upload

- Waldhomeserver is an always-on Ubuntu 24.04 home server (4-core Intel i5, 8GB RAM)

Development (Mac workstation):

- Scheduler policy: **manual only**. Automatic FTPS upload is disabled on developer machines by policy to prevent duplicate production artifacts

- CLI commands used for manual pipeline runs and testing

Cross-host communication:

- File-based handoff: Mac `~/Documents/_agent_comm/outbox/` → SCP → waldhomeserver `~/agent_comm/inbox/`

- Reference: `documentation/05-admin-and-operations/WALD_HOME_SERVER_AGENT_COMMUNICATION.md`

Admin UI

Flask web application at `127.0.0.1:5001` (Hebrew RTL):

Sections:

- **Dashboard** – overview: today's pipeline status, recent runs, alerts, resolution rate
- **Sources** – view/edit source registry, toggle active/inactive, view fetch profiles
- **Products** – browse canonical catalog, edit display settings, view alias coverage
- **Aliases** – manage product alias mappings (raw name → canonical)
- **Scope Skip Rules** – manage the 301 scope-skip rules
- **Observations** – browse normalized observations, apply flags, review unresolvable items
- **Aggregates** – view daily/weekly statistics per product
- **Runs** – pipeline execution history, per-run reports
- **Alerts** – in-app pipeline and operations alerts
- **Publish** – trigger manual publish runs, view artifact history

---

CSS Architecture (Public Site)

Three-layer CSS system for WordPress integration:

1. **Layer 1:** Flatsome parent + child theme (typography, layout, buttons)
2. **Layer 2:** `sfagent-base.css` – shared tokens and components, `.sfagent-*` prefix namespace
3. **Layer 3:** Inline `<style>` in `public_report_body.html` – page-specific rules scoped to `.sfagent-*`

This scoping ensures the report HTML can be embedded via WordPress shortcode without CSS conflicts with the parent theme.

Responsive breakpoint: 640px

- Desktop/tablet (≥640px): Full product table with price metrics in columns
- Mobile (<640px): Stacked product cards, full metrics below each card